Lecture Notes: TwinCAT 3

Real-Time Architecture and "Hello World" Implementation

March 17, 2026

Contents

1 Traditional IT Software vs. PLC Software **2**
- 1.1 Standard OS (Windows) . . . 2
- 1.2 Industrial Automation Requirement . . . 2

2 TwinCAT 3 Architecture: Ring 0 vs. Ring 3 **2**
- 2.1 Ring 0 (Kernel Mode) . . . 2
- 2.2 Ring 3 (User Mode) . . . 2

3 CPU Core Management **3**
- 3.1 Shared Cores . . . 3
- 3.2 Isolated Cores . . . 3

4 Hardware and BIOS Requirements **3**

5 TwinCAT Project Structure **4**
- 5.1 SYSTEM . . . 4
- 5.2 PLC . . . 4
- 5.3 I/O . . . 4
- 5.4 PLC Project Folders . . . 4

6 Real-Time Tasks and Cycle Time **4**

7 Hands-on: "Hello World" with ADSLOGSTR **5**
- 7.1 ADSLOGSTR Function . . . 5
- 7.2 Single-Run Logic . . . 6

8 Licensing and Debugging **9**
- 8.1 Licensing . . . 9
- 8.2 Online Monitoring . . . 9

Quick Reference Card **9**

1 Traditional IT Software vs. PLC Software

The fundamental difference between conventional IT software and PLC software lies in **determinism**.

1.1 Standard OS (Windows)

Uses a **scheduler** to distribute CPU time based on a scheduling policy, such as the "Multilevel feedback queue". This process is **not deterministic**; there are no guarantees on how much CPU time a process gets, and it can be interrupted at any time (context switching).

1.2 Industrial Automation Requirement

Machine control requires strict boundaries for execution time. Interrupting a process while it is controlling a moving object can lead to dangerous results.

Aspect	Standard IT Software	PLC Software
Execution	Non-deterministic	Deterministic
Timing	Best effort	Guaranteed cycle time
Interruption	Can be preempted anytime	Protected execution window
Priority	OS scheduler decides	Fixed, developer-defined
Failure Mode	Slow response	Cycle overrun / fault

Table 1: IT Software vs. PLC Software Characteristics

Warning: Non-deterministic execution in machine control can cause serious safety issues. If a motion control algorithm is interrupted mid-calculation, the resulting commands could cause collisions, damage equipment, or injure personnel.

2 TwinCAT 3 Architecture: Ring 0 vs. Ring 3

TwinCAT 3 acts as a **real-time extension** to the operating system (Windows or Tc/BSD) by running as a **kernel module**. It utilizes the CPU's "rings" to manage priority:

2.1 Ring 0 (Kernel Mode)

Has direct hardware access. This is where the TwinCAT scheduler, real-time tasks, and critical I/O communications (like **EtherCAT**) reside.

2.2 Ring 3 (User Mode)

A restricted space for standard Windows applications (Office, games). Virtual memory prevents these applications from interfering with each other or the system.

Note: The ring architecture is a hardware feature of x86/x64 processors. Ring 0 has the highest privilege level with direct hardware access, while Ring 3 is the most restricted. TwinCAT leverages this to ensure real-time tasks cannot be preempted by user applications.

CPU Protection Rings	
Ring 0 - Kernel Mode	**Ring 3 - User Mode**
TwinCAT Real-Time Kernel EtherCAT Master Real-Time Tasks I/O Drivers	Windows Applications Office, Browsers HMI Applications User Programs
Direct Hardware Access Highest Priority	Virtual Memory Lower Priority

Figure 1: TwinCAT CPU Ring Architecture

3 CPU Core Management

TwinCAT allows developers to distribute workload across CPU cores in two primary ways:

3.1 Shared Cores

TwinCAT and Windows share a core using the **"double-tick" method**.

- The **Base Time** (e.g., 1ms) is a constant interrupt where the scheduler pauses Windows to allow real-time tasks to run.
- There is a limit (typically **90%**) to ensure the OS always has time to execute its own tasks.

3.2 Isolated Cores

A core is exclusively reserved for TwinCAT; Windows "cannot see" this core. This provides **100%** of the core's computing time to real-time tasks, offering high performance and low jitter.

Mode	CPU Usage	Jitter	Use Case
Shared Core	Up to 90%	Higher	General automation
Isolated Core	100%	Minimal	Motion control, high-speed

Table 2: CPU Core Allocation Strategies

Tip: For applications requiring cycle times below 1ms or precise motion control, use isolated cores. For typical PLC logic with cycle times of 10ms or more, shared cores provide adequate performance while maintaining Windows responsiveness.

4 Hardware and BIOS Requirements

To run TwinCAT 3 on a 64-bit machine, **Intel Virtualization Technology (VT-x)** must be enabled in the computer's BIOS. This allows the kernel-mode scheduler to function correctly next to the operating system.

Warning: If Intel VT-x is not enabled, TwinCAT 3 will fail to start or display licensing errors. Access your system BIOS/UEFI settings and ensure virtualization technology is enabled before installing TwinCAT.

Requirement	Setting
Intel VT-x	Enabled
Hyper-Threading	Typically disabled for RT
C-States	Disabled for consistent timing
SpeedStep	Disabled for consistent timing

Table 3: Recommended BIOS Settings for TwinCAT 3

5 TwinCAT Project Structure

A TwinCAT solution is divided into several configurations/modules:

5.1 SYSTEM

Handles real-time settings (core allocation, base time), task definitions, and license management.

5.2 PLC

The primary area for code development.

5.3 I/O

Manages fieldbus communication (e.g., EtherCAT, Profinet) with the outside world.

5.4 PLC Project Folders

- **POUs (Program Organization Units):** Contains the bulk of the code, including function blocks and programs.
- **GVLs (Global Variable Lists):** Used for global variables, which are standard in automation.
- **DUTs (Data Unit Types):** Used to define structures or alias types.

Folder	Contents	Purpose
POUs	Programs, FBs, Functions	Executable code
GVLs	Global variables	Shared data
DUTs	Structures, Enums, Aliases	Custom data types
VISUs	Visualizations	HMI screens

Table 4: PLC Project Folder Structure

6 Real-Time Tasks and Cycle Time

A **TwinCAT Task** is a "time container" for an IEC program, defined by a name, priority, and **cycle time**.

- The **cycle time** specifies the interval at which code is executed.
- Execution is effectively an **endless while-loop**.

- The PLC developer must ensure all code within a task finishes before the cycle time expires to avoid **cycle-time overruns**.

Cycle 1	Cycle 2	Cycle 3	Cycle 4	···
10ms	10ms	10ms	10ms	

Figure 2: Cyclic Task Execution Model

Note: Common cycle times range from 1ms for motion control to 100ms for slow processes like temperature control. Choose the shortest cycle time that meets your application requirements—shorter cycles consume more CPU resources.

7 Hands-on: “Hello World” with ADSLOGSTR

In TwinCAT, printing a message is done via the **ADSLOGSTR()** function, which sends a message from the real-time context to the non-real-time context (the Visual Studio logger or a Windows popup).

7.1 ADSLOGSTR Function

```
// PROGRAM: MAIN
// Demonstrates the ADSLOGSTR function for logging messages
// Messages are sent from RT context to Visual Studio output window

PROGRAM MAIN
VAR
    // Variables for demonstration
    sUserName    : STRING := 'Operator1';
    nErrorCode   : INT := 42;
END_VAR

// ====================================================
// ADSLOGSTR Function Call
// ====================================================
// Syntax: ADSLOGSTR(msgCtrlMask, formatString, stringArgument)
//
// Parameters:
//   msgCtrlMask    : DWORD - Controls message type and destination
//   formatString   : STRING - The message text (can include %s placeholder)
//   stringArgument: STRING - Optional string to replace %s in format
// ====================================================

// Example 1: Simple log message (informational)
// ADSLOG_MSGTYPE_LOG sends to the TwinCAT logger
ADSLOGSTR(
    ADSLOG_MSGTYPE_LOG,            // Message type: Log entry
    'Hello World from TwinCAT!',   // Message format string
    ''                             // No additional argument
);

// Example 2: Log message with string parameter
```

```
// The %s in the format string is replaced by the third argument
ADSLOGSTR(
    ADSLOG_MSGTYPE_LOG,            // Message type: Log entry
    'User logged in: %s',          // Format string with placeholder
    sUserName                      // String argument replaces %s
);

// Example 3: Warning message
// ADSLOG_MSGTYPE_WARN creates a warning-level entry
ADSLOGSTR(
    ADSLOG_MSGTYPE_WARN,           // Message type: Warning
    'Low pressure detected in Zone %s',
    'A3'                           // Zone identifier
);

// Example 4: Error message with popup
// Combining ADSLOG_MSGTYPE_ERROR with ADSLOG_MSGTYPE_LOG
// ADSLOG_MSGTYPE_ERROR can trigger a Windows popup notification
ADSLOGSTR(
    ADSLOG_MSGTYPE_ERROR OR ADSLOG_MSGTYPE_LOG,  // Combined flags
    'Critical fault in module %s - immediate attention required',
    'Conveyor_01'
);

// Example 5: Hint/Information message
ADSLOGSTR(
    ADSLOG_MSGTYPE_HINT,           // Message type: Hint
    'System startup complete at %s',
    'Station_Main'
);
```

Message Type Constant	Value	Purpose
ADSLOG_MSGTYPE_HINT	16#01	Informational hint
ADSLOG_MSGTYPE_WARN	16#02	Warning message
ADSLOG_MSGTYPE_ERROR	16#04	Error message
ADSLOG_MSGTYPE_LOG	16#10	Log to output window
ADSLOG_MSGTYPE_MSGBOX	16#20	Show Windows popup

Table 5: ADSLOGSTR Message Type Constants

7.2 Single-Run Logic

Because the code runs cyclically (e.g., every 10ms), a direct call to `ADSLOGSTR` would flood the logger with 100 messages per second. A **boolean trigger** is required to ensure it only runs once.

```
// PROGRAM: MAIN
// Demonstrates single-run logic to prevent message flooding
// Without this protection, ADSLOGSTR would execute every cycle!

PROGRAM MAIN
VAR
    // ====================================================
    // Boolean flag to ensure code runs only once
    // Initialized to FALSE so the IF block executes on first scan
```

```
    // ====================================================
    bRunOnlyOnce     : BOOL := FALSE;

    // Alternative: Use for triggering on button press
    bTriggerMessage : BOOL := FALSE;
    bTriggerDone     : BOOL := FALSE;

    // For demonstration
    sMessage         : STRING := 'System initialized successfully';
END_VAR

// ====================================================
// SINGLE-RUN PATTERN: Execute Once at Startup
// ====================================================
// This pattern ensures ADSLOGSTR is called exactly once
// when the PLC starts, not every cycle

IF NOT bRunOnlyOnce THEN
    // This block executes only on the first PLC scan
    // because bRunOnlyOnce starts as FALSE

    ADSLOGSTR(
        ADSLOG_MSGTYPE_LOG,
        'Hello World - This message appears only once!',
        ''
    );

    // Additional startup initialization can go here
    ADSLOGSTR(
        ADSLOG_MSGTYPE_LOG,
        'PLC Program Version 1.0 Started',
        ''
    );

    // Set the flag to TRUE to prevent re-execution
    // On all subsequent cycles, the IF condition is FALSE
    bRunOnlyOnce := TRUE;

END_IF

// ====================================================
// TRIGGERED PATTERN: Execute on External Command
// ====================================================
// Use this pattern when you want to log based on a trigger
// such as a button press or event detection

IF bTriggerMessage AND NOT bTriggerDone THEN
    // Execute when trigger goes TRUE (rising edge behavior)

    ADSLOGSTR(
        ADSLOG_MSGTYPE_LOG,
        'Triggered message: %s',
        sMessage
    );

    // Mark as done to prevent re-triggering
    bTriggerDone := TRUE;

```

```
END_IF

// Reset the done flag when trigger is released
// This allows re-triggering when bTriggerMessage goes TRUE again
IF NOT bTriggerMessage THEN
    bTriggerDone := FALSE;
END_IF
```

```
// Advanced Example: Using R_TRIG for edge detection
// This is the professional approach for triggered actions

PROGRAM MAIN
VAR
    bLogButton      : BOOL;              // Input: Button to trigger
    log
    rtLogTrigger    : R_TRIG;            // Rising edge detector
    nMessageCount   : INT := 0;          // Count of messages sent
    sCountStr       : STRING;            // For number formatting
END_VAR

// Rising edge detection - cleaner than manual flag management
rtLogTrigger(CLK := bLogButton);

IF rtLogTrigger.Q THEN
    // This executes exactly once per button press
    // Q is TRUE only on the rising edge of CLK

    nMessageCount := nMessageCount + 1;

    // Convert count to string for logging
    // (In practice, use INT_TO_STRING or format functions)

    ADSLOGSTR(
        ADSLOG_MSGTYPE_LOG,
        'Button pressed - message number logged',
        ''
    );
END_IF
```

Warning: Never call ADSLOGSTR unconditionally in cyclic code! With a 10ms cycle time, you would generate 100 messages per second, quickly filling logs and potentially impacting system performance. Always use single-run logic or edge detection.

Tip: The R_TRIG (rising edge trigger) function block from the Tc2_Standard library is the preferred method for detecting button presses or event transitions. It automatically handles the edge detection logic and is more readable than manual boolean flag management.

8 Licensing and Debugging

8.1 Licensing

Development is **free**. For the runtime, you can generate **7-day trial licenses** indefinitely using a **CAPTCHA**. Permanent licenses are priced based on the **performance class** of the target PLC.

License Type	Duration	Use Case
Development	Unlimited	XAE on dev PC
Trial	7 days (renewable)	Testing, evaluation
Production	Permanent	Deployed systems

Table 6: TwinCAT 3 License Types

8.2 Online Monitoring

One of TwinCAT's most powerful features is the ability to **"login"** to the PLC. This allows developers to:

- Watch variable values in real-time
- Change data on-the-fly
- "Force" values to debug logic without restarting the system

> **Note:** Online monitoring is invaluable for commissioning and troubleshooting. You can observe actual I/O states, step through logic mentally by watching variables change, and temporarily override values to test different scenarios—all without stopping the PLC.

Quick Reference Card

CPU Rings

Ring	Mode	Contents
Ring 0	Kernel	TwinCAT RT, EtherCAT
Ring 3	User	Windows apps, HMI

Project Structure

ADSLOGSTR Quick Reference

```
// Basic usage
ADSLOGSTR(ADSLOG_MSGTYPE_LOG, 'Message', '');

// With parameter
ADSLOGSTR(ADSLOG_MSGTYPE_LOG, 'Value: %s', sParam);

// Single-run pattern
```

Folder	Purpose
SYSTEM	RT settings, tasks, licenses
PLC	Code development
I/O	Fieldbus configuration
POUs	Programs, FBs, Functions
GVLs	Global variables
DUTs	Custom data types

```
IF NOT bDone THEN
    ADSLOGSTR(...);
    bDone := TRUE;
END_IF
```

Message Type Flags

Constant	Effect
ADSLOG_MSGTYPE_LOG	Log to output
ADSLOG_MSGTYPE_WARN	Warning level
ADSLOG_MSGTYPE_ERROR	Error level
ADSLOG_MSGTYPE_MSGBOX	Windows popup

Last Updated: March 17, 2026